UTEC
Universidad Tecnológica

Singleton- conexión a la base de datos

Paso 1: La base de datos

Hay 2 opciones: o utilizan una instalación local de postgresql, o utilizan un docker con la base de datos, de modo de no estar instalando cosas en sus máquinas.

Ver anexo Docker

Si vamos por la opción de usar un docker…

Crear el archivo `docker-compose.yml`

Crea un archivo llamado `docker-compose.yml` en la raíz de tu proyecto. Este archivo definirá el servicio de la base de datos PostgreSQL.

```
version: '3.8'

services:
  postgres:
    image: postgres:16.2
    container_name: mi_postgres
    environment:
      POSTGRES_USER: usuario
      POSTGRES_PASSWORD: contraseña
      POSTGRES_DB: mi_base_de_datos
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Paso 2: Crear el proyecto en IntelliJ

1. **Abrir IntelliJ IDEA** y seleccionar "New Project".
2. Elegir "Maven" y marcar "Create from archetype".

UTEC
Universidad Tecnológica

3. Completar los detalles del proyecto, como el `GroupId`, `ArtifactId` y la versión de Java. Utilizar el archetype **maven-archetype-quickstart**: Este archetype crea una estructura de proyecto básica que incluye una clase principal y un archivo de prueba. Es ideal para comenzar.
4. Hacer clic en "Finish" para crear el proyecto.

Paso 3: Configurar el `pom.xml`

Abre el archivo `pom.xml` y agrega las dependencias necesarias para el controlador de PostgreSQL y el proyecto. Asegúrate de que tu archivo `pom.xml` tenga al menos lo siguiente:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.ejemplo</groupId>
    <artifactId>mi-proyecto-postgres</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <version>42.6.0</version> <!-- Versión del driver de
PostgreSQL -->
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.8.1</version>
```

UTEC
Universidad Tecnológica

```
                <configuration>
                    <source>17</source>
                    <target>17</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Paso 4: Implementar el Singleton para la conexión a PostgreSQL

Crea una clase llamada `ConexionBDSingleton` en tu proyecto. Esta clase será responsable de manejar la conexión a la base de datos usando el patrón Singleton.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionBDSingleton {
    private static ConexionBDSingleton instanciaUnica;
    private Connection conexion;
    private String url =
"jdbc:postgresql://localhost:5432/mi_base_de_datos";
    private String usuario = "usuario";
    private String contraseña = "contraseña";

    private ConexionBDSingleton() throws SQLException {
        this.conexion = DriverManager.getConnection(url, usuario,
contraseña);
    }

    public static ConexionBDSingleton obtenerInstancia() throws
SQLException {
        if (instanciaUnica == null) {
            instanciaUnica = new ConexionBDSingleton();
        }
        return instanciaUnica;
    }
```

UTEC
Universidad Tecnológica

```
    public Connection getConexion() {
        return conexion;
    }
}
```

Paso 5: Crear la clase principal

Crea una clase principal para probar la conexión a la base de datos:

```
public class Main {
    public static void main(String[] args) {
        try {
            ConexionBDSingleton conexion =
ConexionBDSingleton.obtenerInstancia();
            System.out.println("Conexión a la base de datos establecida:
" + (conexion.getConexion() != null));
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Paso 6: Levantar el contenedor de PostgreSQL – SOLO PARA DOCKER

Desde la terminal, navega hasta la carpeta donde se encuentra tu archivo `docker-compose.yml` y ejecuta el siguiente comando:

```
docker-compose up
```

Esto descargará la imagen de PostgreSQL y levantará el contenedor. Asegúrate de que el contenedor esté corriendo antes de probar la conexión.

Paso 7: Ejecutar tu proyecto

1. En IntelliJ, asegúrate de que el proyecto esté correctamente configurado.
2. Ejecuta la clase `Main` para verificar que se establezca la conexión a la base de datos.

UTEC
Universidad Tecnológica

Docker

¿Qué es Docker?

Docker es una plataforma que permite crear, desplegar y gestionar aplicaciones en contenedores. Un **contenedor** es una unidad estandarizada de software que empaqueta el código de una aplicación y todas sus dependencias para que se ejecute de manera rápida y confiable en diferentes entornos.

Características principales de Docker:

- **Aislamiento**: Los contenedores funcionan de manera aislada, lo que significa que cada aplicación puede tener su propio entorno, sin interferir con otras.
- **Portabilidad**: Los contenedores pueden ejecutarse en cualquier sistema que tenga Docker instalado, sin importar el sistema operativo subyacente.
- **Eficiencia**: A diferencia de las máquinas virtuales, los contenedores comparten el mismo núcleo del sistema operativo, lo que los hace más ligeros y rápidos de iniciar.

¿Cómo instalar Docker?

La instalación de Docker varía según el sistema operativo.

Instalar Docker en Windows

- Abre tu navegador web y visita https://www.docker.com/products/docker-desktop/.
- Descarga el instalador de Docker Desktop para Windows y ejecútalo.
- Sigue las instrucciones del instalador para completar la instalación de Docker Desktop en tu máquina.
- Una vez que la instalación haya finalizado, abre una ventana de línea de comandos (Command Prompt) o PowerShell.
- Ejecuta el siguiente comando para verificar que Docker Compose se haya instalado correctamente:
  docker-compose --version

Si el comando muestra la versión de Docker Compose instalada quedó todo pronto!

UTEC
Universidad Tecnológica

Comandos básicos de Docker

1. Verificar la versión de Docker

Para comprobar que Docker está instalado correctamente y ver la versión, usa:

```
docker --version
```

2. Levantar un contenedor

Para ejecutar una aplicación en un contenedor, puedes usar la imagen de Docker de una aplicación. Por ejemplo, para levantar un contenedor de Nginx, usa:

```
docker run -d -p 80:80 nginx
```

- `-d`: Ejecuta el contenedor en segundo plano (detached mode).
- `-p 80:80`: Mapea el puerto 80 del contenedor al puerto 80 de tu máquina.

3. Listar contenedores en ejecución

Para ver los contenedores que están actualmente en ejecución, utiliza:

```
docker ps
```

4. Detener un contenedor

Si necesitas detener un contenedor en ejecución, primero necesitas su ID o nombre, que puedes obtener con `docker ps`. Luego ejecuta:

```
docker stop <nombre_o_id_del_contenedor>
```

5. Eliminar un contenedor

Para eliminar un contenedor (detenido o en ejecución), usa:

```
docker rm <nombre_o_id_del_contenedor>
```

UTEC
Universidad Tecnológica

6. Ver imágenes de Docker

Para ver las imágenes que tienes en tu sistema, usa:

```
docker images
```

7. Eliminar una imagen

Para eliminar una imagen que ya no necesites, utiliza:

```
docker rmi <nombre_o_id_de_la_imagen>
```

Repositorio GIT del ejemplo

https://git.utec.edu.uy/pa-2024/pa-2024-s2-singleton